

Dendê Eventos (Refatorando o Código)

Equipe: Londres

Docente: Lucas Almeida Silva

Disciplina: Desenvolvimento Mobile Nativo

Sobre a apresentação

- Introdução

A apresentação irá demonstrar a segunda (2ª) etapa de uma atividade realizada no Centro Universitário de Excelência (UNEX).

- Objetivo

Apresentar o código (refatorado) em Kotlin realizado para o aplicativo “Dendê Eventos” da empresa fictícia Dendê Softhouse.

Agenda

Tópico 1: Funções refatoradas

Explicar como funciona a respectiva User Story da função no novo modelo.

Tópico 2: Estrutura de arquivos final

Apresentar a estrutura de arquivos final do projeto.

Tópico 3: Model.kt

Apresentar o arquivo que agrupa data/enum classes essenciais para o funcionamento do sistema.

Tópico 4: Operações do repositório

Explicar o papel que cada função que consta em Repositorio.kt desempenha.

Tópico 5: Funções de componente e dois formulários criados com elas

Explicar o papel que cada função que consta em Components.kt desempenha e a aplicação de duas (2) delas em formulários.

Tópico 6: Resultado em Main.kt

Apresentação de resultado do fluxo principal após a atualização do sistema.

Tópico 1: Funções refatoradas

- Objetivo

A refatoração de código faz-se necessária quando há otimizações a serem exploradas em um programa, portanto, o Dendê Eventos passou por uma mudança total de forma, incluindo: novas funções (reduzindo laços de repetição), novos arquivos separando-as e uma 'main' mais fluida.

Tópico 1: Funções a serem apresentadas

1. Alterar Perfil do Usuário

A função Alterar Perfil do Usuário permite que o usuário faça alterações às informações fornecidas durante o cadastro no sistema ou já alteradas anteriormente.

```
data class DadosUsuario(  
    var statusConta: Boolean,  
    var nome: String,  
    var dataNascimento: String,  
    var sexo: SexoUsuario,  
    val email: String,  
    var senha: String,  
    val tipoUsuario: TipoUsuario,  
    var cnpj: String? = null,  
    var razaoSocial: String? = null,  
    var nomeFantasia: String? = null  
)
```

*statusConta, email e tipoUsuario não disponíveis para modificar na função alterarUsuario

(Tópico 1) Função: Alterar Perfil do Usuário

```
when (opcaoAlterarUsuario) {  
    0 -> menuAlterarUsuario = false  
    1 -> {  
        usuarioEncontrado.nome = readString( mensagem = "Digite nome atualizado: ",  
            mensagemErro = "ERR0: Nome inválido. Tente novamente.",  
            tamanhoMinimo = 2).uppercase()  
        println("OK: Nome atualizado para ${usuarioEncontrado.nome}.")  
    }  
    2 -> {  
        println("Alterar data de nascimento:")  
        val diaNascimento = readInt( mensagem = "Digite dia atualizado: ",  
            mensagemErro = "ERR0: Dia inválido. Tente novamente.", intervalo = 1 ≤ .. ≤ 31)  
        val mesNascimento = readInt( mensagem = "Digite mês atualizado: ",  
            mensagemErro = "ERR0: Mês inválido. Tente novamente.", intervalo = 1 ≤ .. ≤ 12)  
        val anoNascimento = readInt( mensagem = "Digite ano atualizado: ",  
            mensagemErro = "ERR0: Ano inválido. Tente novamente.", intervalo = 1920 ≤ .. ≤ 2020)  
        usuarioEncontrado.dataNascimento = "$diaNascimento/$mesNascimento/$anoNascimento"  
        println("OK: Data de nascimento atualizada para ${usuarioEncontrado.dataNascimento}.")  
    }  
}
```

- Descrição Imagem
- When para números
- Atualizar nome
(maior que 2 dígitos;
caixa alta padrão)
- Atualizar data de
nascimento (com
validação padrão
para números
dia/mês/ano)

(Tópico 1) Função: Alterar Perfil do Usuário

```
3 -> {
    println("Sexo atualizado: \n1. Masculino | 2. Feminino | 3. Não informar")
    val novaOpcaoSexo = readInt(mensagem = "Digite opção: ",
        mensagemErro = "ERRO: Opção inválida. Tente de novo.", intervalo = 1 ≤ .. ≤ 3)
    usuarioEncontrado.sexo = when (novaOpcaoSexo) {
        1 -> SexoUsuario.MASCULINO
        2 -> SexoUsuario.FEMININO
        else -> SexoUsuario.NAO_INFORMADO
    }
    println("OK: Sexo atualizado para ${usuarioEncontrado.sexo}.")
}

4 -> {
    val senhaInserida = readString(mensagem = "Digite nova senha: ", mensagemErro = "ERRO: Senha
    val senhaConfirmada = readString(mensagem = "Digite nova senha: ",
        mensagemErro = "ERRO: Senha deve conter 8 caracteres. Tente de novo.", tamanhoMinimo = 8)
    senhaValida(senhaInserida, senhaConfirmada = senhaConfirmada)
    usuarioEncontrado.senha = senhaConfirmada
    println("OK: Senha atualizada.")
}
```

- Desc. Imagem
- Atualizar sexo com intervalo de opções
- Atualizar senha com limites de 8 dígitos mínimos, conferida 2 vezes

(Tópico 1) Função: Alterar Perfil do Usuário

```
5 -> {
    usuarioEncontrado.cnpj = readString( mensagem = "CNPJ atualizado: ",
        mensagemErro = "CNPJ inválido. Digite 14 números.", tamanhoMinimo = 14)
    println("OK: CNPJ atualizado para ${usuarioEncontrado.cnpj}.")
}

6 -> {
    usuarioEncontrado.razaoSocial = readString( mensagem = "Razão social atualizada: ",
        mensagemErro = "Campo obrigatório. Tente novamente.").uppercase()
    println("OK: Razão social atualizada para ${usuarioEncontrado.razaoSocial}.")
}

7 -> {
    usuarioEncontrado.nomeFantasia = readString( mensagem = "Nome fantasia atualizado: ",
        mensagemErro = "Campo obrigatório. Tente novamente.").uppercase()
    println("OK: Nome fantasia atualizado para ${usuarioEncontrado.nomeFantasia}.")
}
} while (menuAlterarUsuario)
```

- Desc. Imagem
- Atualizar CNPJ (14 dígitos)
- Atualizar razão social e nome fantasia (caixa alta padrão)
- Final de loop do-while do menu


```

fun alterarUsuario(usuarioEncontrado: DadosUsuario) {
    var menuAlterarUsuario = true
    do {
        println("\nAlterar cadastro de ${usuarioEncontrado.email}")
        println("1. Nome | 2. Data de nascimento | 3. Sexo | 4. Senha")

        when {
            usuarioEncontrado.tipoUsuario == TipoUsuario.ORGANIZADOR ->
                println("5. CNPJ | 6. Razão social | 7. Nome fantasia")
        }
        println("0. Voltar")

        val limiteOpcoes = when (usuarioEncontrado.tipoUsuario) {
            TipoUsuario.ORGANIZADOR -> 0 ≤ .. ≤ 7
            else -> 0 ≤ .. ≤ 4
        }

        val opcaoAlterarUsuario = readInt( mensagem = "Digite opção: ",
            mensagemErro = "ERRO: Opção inválida. Tente novamente.",
            intervalo = limiteOpcoes)

        when (opcaoAlterarUsuario) {
            0 -> menuAlterarUsuario = false
            1 -> {
                usuarioEncontrado.nome = readString( mensagem = "Digite nome atualizado: ",
                    mensagemErro = "ERRO: Nome inválido. Tente novamente.",
                    tamanhoMinimo = 2).uppercase()
                println("OK: Nome atualizado para ${usuarioEncontrado.nome}.")
            }
            2 -> {
                println("Alterar data de nascimento:")
                val diaNascimento = readInt( mensagem = "Digite dia atualizado: ",
                    mensagemErro = "ERRO: Dia inválido. Tente novamente.", intervalo = 1 ≤ .. ≤ 31)
                val mesNascimento = readInt( mensagem = "Digite mês atualizado: ",
                    mensagemErro = "ERRO: Mês inválido. Tente novamente.", intervalo = 1 ≤ .. ≤ 12)
                val anoNascimento = readInt( mensagem = "Digite ano atualizado: ",
                    mensagemErro = "ERRO: Ano inválido. Tente novamente.", intervalo = 1920 ≤ .. ≤ 2020)
                usuarioEncontrado.dataNascimento = "$diaNascimento/$mesNascimento/$anoNascimento"
                println("OK: Data de nascimento atualizada para ${usuarioEncontrado.dataNascimento}.")
            }
        }
    }
}

```

```

3 -> {
    println("Sexo atualizado: \n1. Masculino | 2. Feminino | 3. Não informar")
    val novaOpcaoSexo = readInt( mensagem = "Digite opção: ",
        mensagemErro = "ERRO: Opção inválida. Tente de novo.", intervalo = 1 ≤ .. ≤ 3)
    usuarioEncontrado.sexo = when (novaOpcaoSexo) {
        1 -> SexoUsuario.MASCULINO
        2 -> SexoUsuario.FEMININO
        else -> SexoUsuario.NAO_INFORMADO
    }
    println("OK: Sexo atualizado para ${usuarioEncontrado.sexo}.")
}

4 -> {
    val senhaInserida = readString( mensagem = "Digite nova senha: ", mensagemErro = "ERRO: Senha inválida. Tente de novo.")
    val senhaConfirmada = readString( mensagem = "Digite nova senha: ",
        mensagemErro = "ERRO: Senha deve conter 8 caracteres. Tente de novo.", tamanhoMinimo = 8)
    senhaValida(senhaInserida, senhaConfirmada = senhaConfirmada)
    usuarioEncontrado.senha = senhaConfirmada
    println("OK: Senha atualizada.")
}

5 -> {
    usuarioEncontrado.cnpj = readString( mensagem = "CNPJ atualizado: ",
        mensagemErro = "CNPJ inválido. Digite 14 números.", tamanhoMinimo = 14)
    println("OK: CNPJ atualizado para ${usuarioEncontrado.cnpj}.")
}

6 -> {
    usuarioEncontrado.razaoSocial = readString( mensagem = "Razão social atualizada: ",
        mensagemErro = "Campo obrigatório. Tente novamente.").uppercase()
    println("OK: Razão social atualizada para ${usuarioEncontrado.razaoSocial}.")
}

7 -> {
    usuarioEncontrado.nomeFantasia = readString( mensagem = "Nome fantasia atualizado: ",
        mensagemErro = "Campo obrigatório. Tente novamente.").uppercase()
    println("OK: Nome fantasia atualizado para ${usuarioEncontrado.nomeFantasia}.")
}
} while (menuAlterarUsuario)

```



Tópico 1: Funções a serem apresentadas

1. Alterar Perfil do Usuário

2. Alterar Evento

A função Alterar Evento permite que o usuário organizador faça alterações às informações fornecidas durante o cadastro de um evento no sistema ou já alteradas anteriormente.

```
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```

*id, organizadorEmail e statusEvento não disponíveis para modificar na função alterarEvento

(Tópico 1) Função: Alterar Evento

- Ainda não
refatorada

Tópico 1: Funções a serem apresentadas

1. Alterar Perfil do Usuário
2. Alterar Evento
3. Listar eventos do organizador

A função Listar eventos do organizador permite que o usuário organizador veja todos os cadastros de seus eventos no sistema.

```
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```

*vários dados do data class DadosEvento são exibidos com o uso da função listarEventosOrganizador

(Tópico 1) Fun: Listar eventos do organizador

- Ainda não
refatorada

Tópico 1: Funções a serem apresentadas

1. Alterar Perfil do Usuário
2. Alterar Evento
3. Listar eventos do organizador
4. Listar ingressos

A função Listar ingressos permite que o usuário comum veja todos os seus ingressos obtidos no sistema.

```
data class DadosIngresso(  
    val id: Int,  
    val idEvento: Int,  
    val emailUsuario: String,  
    var statusDisponibilidade: Boolean,  
    val valorPago: Double  
)  
  
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```

*vários dados do data classes DadosIngresso e DadosEvento são exibidos com o uso da função listarIngressosUsuario

(Tópico 1) Função: Listar ingressos

- Ainda não refatorada

Tópico 2: Estrutura de arquivos final

- Objetivo

Parte de uma refatoração de código em um sistema abrange uma nova distribuição de métodos/funções em diferentes arquivos em seu diretório. Graças às novas funções criadas para cobrir cada User Story ou laço de repetição (loop) do fluxo principal (main), fez-se necessário uma separação de código para que se tornasse mais dinâmico o entendimento do que cada coisa faz e para qual objetivo faz.

Tópico 2: Arquivos a serem apresentados

1. Model.kt

Este arquivo .kt (Kotlin) guarda todos os data classes e enums criados no sistema.

São eles: SexoUsuario (enum), TipoUsuario (enum), DadosUsuario, TipoEvento (enum), ModalidadeEvento (enum), DadosEvento, DadosIngresso

```
enum class SexoUsuario { MASCULINO, FEMININO, NAO_INFORMADO }
```

13 Usos

```
enum class TipoUsuario { COMUM, ORGANIZADOR }
```

18 Usos

```
data class DadosUsuario(  
    var statusConta: Boolean,  
    var nome: String,  
    var dataNascimento: String,  
    var sexo: SexoUsuario,  
    val email: String,  
    var senha: String,  
    val tipoUsuario: TipoUsuario,  
    var cnpj: String? = null,  
    var razaoSocial: String? = null,  
    var nomeFantasia: String? = null  
)
```

25 Usos

```
enum class TipoEvento {  
    1 Uso  
    SOCIAL, CORPORATIVO, ACADEMICO, CULTURAL_ENTRETENIMENTO, RELIGIOSO, ESPORTIVO,  
    1 Uso  
    FEIRA, CONGRESSO, OFICINA, CURSO, TREINAMENTO, AULA, SEMINARIO, PALESTRA, SHOW,  
    1 Uso  
    FESTIVAL, EXPOSICAO, RETIRO, CULTO, CELEBRACAO, CAMPEONATO, CORRIDA, OUTRO
```

```
}
```

5 Usos

```
enum class ModalidadeEvento { PRESENCIAL, REMOTO, HIBRIDO }
```

16 Usos

```
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```

```
data class DadosIngresso(  
    val id: Int,  
    val idEvento: Int,  
    val emailUsuario: String,  
    var statusDisponibilidade: Boolean,  
    val valorPago: Double
```

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt

Este arquivo .kt (Kotlin) guarda todas as funções criadas para definir regras à leitura de dados de formulários no sistema e exibir tabelas baseadas em listas.

São elas: readInt (números inteiros), readDouble (números racionais), readString (texto), printTable (listas)

```

fun readInt(mensagem: String,
            mensagemErro: String,
            intervalo: IntRange = 0..Int.MAX_VALUE): Int {
    var valorFinal: Int? = null

    do {
        print(mensagem)
        when (val entradaUsuario = readln().toIntOrNull()) {
            null -> println(mensagemErro)
            in intervalo -> valorFinal = entradaUsuario
            else -> println(mensagemErro)
        }
    } while (valorFinal == null)

    return valorFinal
}

fun readDouble(mensagem: String,
               mensagemErro: String,
               valorMinimo: Double=0.0,
               valorMaximo: Double=Double.MAX_VALUE): Double {
    var valorFinal: Double? = null

    do {
        print(mensagem)
        when (val entradaUsuario = readln().toDoubleOrNull()) {
            null -> println(mensagemErro)
            in valorMinimo..valorMaximo -> valorFinal = entradaUsuario
            else -> println(mensagemErro)
        }
    } while (valorFinal == null)

    return valorFinal
}

```

```

fun readString(mensagem: String, mensagemErro: String, tamanhoMinimo: Int = 1): String {
    var valorFinal: String? = null

    do {
        print(mensagem)
        val entradaUsuario = readln()

        when {
            entradaUsuario.length >= tamanhoMinimo -> valorFinal = entradaUsuario
            else -> println(mensagemErro)
        }
    } while (valorFinal == null)

    return valorFinal
}

fun printTable(cabecalho: String, colunas: List<String>, linhas: List<List<String?>>) {
    println("\n$cabecalho")

    val larguraColunas = colunas.mapIndexed { index, title ->
        val largurasLinhas = linhas.maxOfOrNull { it[index]?.length ?: 0 } ?: 0
        maxOf( a = title.length, b = largurasLinhas)
    }

    val separadores = larguraColunas.joinToString( separator = "-+-", prefix = "-+-", postfix = "-+-" ) { "-".repeat( n = it ) }
    val linhaCabecalho = colunas.mapIndexed { index, title ->
        title.padEnd( length = larguraColunas[index])
    }.joinToString( separator = " | ", prefix = " | ", postfix = " | ")

    println(separadores); println(linhaCabecalho); println(separadores)

    linhas.forEach { row ->
        val linhaFormatada = row.mapIndexed { index, cell ->
            cell?.padEnd( length = larguraColunas[index]) ?: 0
        }.joinToString( separator = " | ", prefix = " | ", postfix = " | ")
        println(linhaFormatada)
    }
    println(separadores)
}

```

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt
3. Repositorio.kt

Este objeto .kt (Kotlin) define o banco de dados do sistema, que tem o objetivo de salvar data, usuários, eventos e ingressos, e funções relacionadas a buscas neste banco de dados.

São algumas: definirDataHoje, adicionarUsuario, buscarUsuarioLogin, possuiEventos, adicionarEvento, buscarEventold, proximoIngressold, ordenarIngressosUsuario...


```

object Repositorio {
    3 Usos
    val listaUsuarios = mutableListOf<DadosUsuario>()
    8 Usos
    val listaEventos = mutableListOf<DadosEvento>()
    7 Usos
    val listaIngressos = mutableListOf<DadosIngresso>()
    5 Usos
    var diaHoje = 0
    5 Usos
    var mesHoje = 0
    5 Usos
    var anoHoje = 0
    8 Usos
    var dataHoje = 0

```

```

// Métodos para datas

```

```

2 Usos
fun digitarDataHoje() {...}

```

```

1 Uso
fun definirDataHoje(diaInserido: Int, mesInserido: Int, anoInserido: Int) {...}
// Métodos para usuários

```

```

2 Usos
fun adicionarUsuario(usuario: DadosUsuario) {...}

```

```

1 Uso
fun buscarUsuarioEmail(email: String): DadosUsuario? {...}

```

```

1 Uso
fun buscarUsuarioLogin(email: String, senha: String): DadosUsuario? {...}

```

```

1 Uso
fun possuiEventos(usuarioEncontrado: DadosUsuario): Boolean {...}

```

```

// Métodos para eventos

```

```

1 Uso
fun adicionarEvento(evento: DadosEvento) {...}

```

```

21 Usos
fun buscarEventoId(id: Int): DadosEvento? {...}

```

```

1 Uso
fun buscarEventosDisponiveis(): List<DadosEvento> {...}

```

```

2 Usos
fun buscarEventosOrganizador(email: String): List<DadosEvento> {...}

```

```

fun ordenarEventosOrganizador(email: String): List<DadosEvento> {...}
// Métodos para ingressos

```

```

3 Usos
fun adicionarIngresso(ingresso: DadosIngresso) {...}

```

```

1 Uso
fun buscarIngressoId(id: Int): DadosIngresso? {...}

```

```

1 Uso
fun buscarIngressosUsuario(email: String): List<DadosIngresso> {...}

```

```

3 Usos
fun proximoIngressoId(): Int {...}

```

```

1 Uso
fun ordenarIngressosUsuario(email: String): List<DadosIngresso> {...}

```

```

fun buscarIngressosValidosEvento(idEvento: Int): List<DadosIngresso> {...}

```

```

3 Usos
fun contarIngressosVendidos(idEvento: Int): Int {...}

```

```

fun desativarIngressosDoEvento(idEvento: Int): Pair<Int, Double> {...}

```

Dendê Eventos (Refatorando o Código)

unex

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt
3. Repositorio.kt
4. Usuario.kt

Este objeto .kt (Kotlin) define funções relacionadas às User Stories de um usuário.

São elas: loginUsuario, emailValido, senhaValida, cadastrarUsuario, visualizarUsuario, alterarUsuario, desativarUsuario...

object Usuario {

1 Uso

fun loginUsuario(): DadosUsuario? {...}

1 Uso

fun emailValido(emailInserido: String, emailConfirmado: String): Boolean {...}

2 Usos

fun senhaValida(senhaInserida: String, senhaConfirmada: String): Boolean {...}

1 Uso

fun cadastrarUsuario() {...}

1 Uso

fun visualizarUsuario(usuarioEncontrado: DadosUsuario) {...}

1 Uso

fun alterarUsuario(usuarioEncontrado: DadosUsuario) {...}

1 Uso

fun desativarUsuario(usuarioEncontrado: DadosUsuario) {...}

}

Dendê Eventos (Refatorando o Código)

unex

27

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt
3. Repositorio.kt
4. Usuario.kt
5. Evento.kt

Este objeto .kt (Kotlin) define funções relacionadas às User Stories e ações com um evento.

São elas: visualizarEvento, feedEventos, cadastrarEvento...

```
object Evento {  
    1 Uso  
    fun visualizarEvento(usuarioEncontrado: DadosUsuario) {...}  
  
    1 Uso  
    fun feedEventos(usuarioEncontrado: DadosUsuario) {...}  
  
    1 Uso  
    fun cadastrarEvento(usuarioEncontrado: DadosUsuario) {...}  
}
```

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt
3. Repositorio.kt
4. Usuario.kt
5. Evento.kt
6. Ingresso.kt

Este objeto .kt (Kotlin) define funções relacionadas às User Stories e ações com um ingresso. São elas: comprarIngresso, cancelarIngresso, visualizarIngressos...

```
object Ingresso {
```

```
    1 Uso
```

```
    fun comprarIngresso(eventoDetalhes: DadosEvento, usuarioEncontrado: DadosUsuario) {...}
```

```
    1 Uso
```

```
    fun cancelarIngresso(ingressoExpandido: DadosIngresso, eventoDoIngresso: DadosEvento) {...}
```

```
    1 Uso
```

```
    fun visualizarIngressos(usuarioEncontrado: DadosUsuario) {...}
```

```
}
```

Tópico 2: Arquivos a serem apresentados

1. Model.kt
2. Components.kt
3. Repositorio.kt
4. Usuario.kt
5. Evento.kt
6. Ingresso.kt
7. Main.kt

Este arquivo .kt (Kotlin) define o fluxo principal do programa. É na função *main* que a interação do cliente do serviço com menus, formulários é feita. Menus como: inicial (opções de login, cadastro e alteração de data), logado (opções de visualizar perfil, alterar cadastro, desativar conta, etc).


```

fun main() {
    println("Dendê Eventos\nby Dendê Softhouse\n")

    Repositorio.digitarDataHoje()

    var opcaoMenuInicial = -1
    do {
        when (opcaoMenuInicial) {
            1 -> {
                val usuarioEncontrado = Usuario.loginUsuario()
                var areaLogada = true
                var opcaoMenuUsuario = -1

                do {
                    when {
                        usuarioEncontrado != null && usuarioEncontrado.statusConta -> {
                            println("\nOlá, ${usuarioEncontrado.nome}.")

                            when (opcaoMenuUsuario) {
                                0 -> {
                                    areaLogada = false
                                }
                            }
                        }
                    }
                    println("OK: Sessão encerrada.")
                }

                1 -> {
                    Usuario.visualizarUsuario(usuarioEncontrado)
                }

                2 -> {
                    Usuario.alterarUsuario(usuarioEncontrado)
                }

                3 -> {
                    Usuario.desativarUsuario(usuarioEncontrado)
                    when {
                        !usuarioEncontrado.statusConta -> areaLogada = false
                    }
                }

                4 -> {
                    Evento.feedEventos(usuarioEncontrado)
                }
            }
        }
    }
}

```

```

5 -> {
    Ingresso.visualizarIngressos(usuarioEncontrado)
}

6 -> {
    Evento.cadastrarEvento(usuarioEncontrado)
}

7 -> {
}

else -> println("Selecione uma das opções visíveis.")
}

println("Menu do usuário (${Repositorio.diaHoje}/${Repositorio.mesHoje}/${Repositorio.anoHoje})")
println("1. Visualizar perfil | 2. Alterar perfil | 3. Desativar perfil")
when (usuarioEncontrado.tipoUsuario) {
    TipoUsuario.COMUM -> println("4. Acessar feed de eventos | 5. Visualizar ingressos")
    TipoUsuario.ORGANIZADOR -> println("6. Criar evento | 7. Visualizar eventos | 8. Alterar eventos")
}
println("0. Sair da conta (${usuarioEncontrado.email})")
opcaoMenuUsuario = readInt(mensagem = "Digite opção: ", mensagemErro = "ERRO: ")

else -> areaLogada = false
}
} while (areaLogada)

2 -> {
    Usuario.cadastrarUsuario()
}

3 -> {
    Repositorio.digitarDataHoje()
}

println("Menu do sistema (${Repositorio.diaHoje}/${Repositorio.mesHoje}/${Repositorio.anoHoje})")
println("1. Acessar usuário")
println("2. Cadastrar usuário")
println("3. Ajustar data do sistema")
println("0. Sair do sistema")
}

```

Tópico 3: Model.kt

- Objetivo

Como mencionado anteriormente, o Model.kt guarda todos os data classes e enums criados no sistema. Estes, são essenciais para o banco de dados do sistema, devido a necessidade de guardar informações em campos predefinidos de elementos predefinidos.

Exemplo: Sabe-se que um usuário de qualquer serviço de rede social precisa de um cadastro com ao menos “email” e “senha”. Felizmente, data class define justamente estes campos que estão em todos os “usuários”.

Tópico 3: Enums a serem apresentados

1. SexoUsuario

O enum class SexoUsuario guarda a predefinição de três (3) opções para o campo “sexo” de um usuário.

```
enum class SexoUsuario { MASCULINO, FEMININO, NAO_INFORMADO }
```

Tópico 3: Enums a serem apresentados

1. SexoUsuario

2. TipoUsuario

O enum class TipoUsuario guarda a predefinição de duas (2) opções para o campo “tipoUsuario” de um usuário.

```
enum class TipoUsuario { COMUM, ORGANIZADOR }
```

Tópico 3: Enums a serem apresentados

1. SexoUsuario

2. TipoUsuario

3. TipoEvento

O enum class TipoEvento guarda a predefinição de vinte e três (23) opções para o campo “tipo” de um evento.

```
enum class TipoEvento {  
    1 Uso  
    SOCIAL, CORPORATIVO, ACADEMICO, CULTURAL_ENTRETENIMENTO, RELIGIOSO, ESPORTIVO,  
    1 Uso  
    FEIRA, CONGRESSO, OFICINA, CURSO, TREINAMENTO, AULA, SEMINARIO, PALESTRA, SHOW,  
    1 Uso  
    FESTIVAL, EXPOSICAO, RETIRO, CULTO, CELEBRACAO, CAMPEONATO, CORRIDA, OUTRO  
}
```

Tópico 3: Enums a serem apresentados

1. SexoUsuario
2. TipoUsuario
3. TipoEvento
4. ModalidadeEvento

O enum class ModalidadeEvento guarda a predefinição de três (3) opções para o campo “ModalidadeEvento” de um evento.

```
enum class ModalidadeEvento { PRESENCIAL, REMOTO, HIBRIDO }
```

Tópico 3: Data classes a serem apresentados

1. DadosUsuario

O data class DadosUsuario guarda a predefinição de dez (10) dados de um usuário.

```
data class DadosUsuario(  
    var statusConta: Boolean,  
    var nome: String,  
    var dataNascimento: String,  
    var sexo: SexoUsuario,  
    val email: String,  
    var senha: String,  
    val tipoUsuario: TipoUsuario,  
    var cnpj: String? = null,  
    var razaoSocial: String? = null,  
    var nomeFantasia: String? = null  
)
```

Tópico 3: Data classes a serem apresentados

1. DadosUsuario
2. DadosEvento

O data class DadosEvento guarda a predefinição de vinte e quatro (24) dados de um evento.

```
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```


Tópico 3: Data classes a serem apresentados

1. DadosUsuario
2. DadosEvento
3. DadosIngresso

O data class DadosIngresso guarda a predefinição de cinco (5) dados de um ingresso.

```
data class DadosIngresso(  
    val id: Int,  
    val idEvento: Int,  
    val emailUsuario: String,  
    var statusDisponibilidade: Boolean,  
    val valorPago: Double  
)
```

```
enum class SexoUsuario { MASCULINO, FEMININO, NAO_INFORMADO }
```

13 Usos

```
enum class TipoUsuario { COMUM, ORGANIZADOR }
```

18 Usos

```
data class DadosUsuario(  
    var statusConta: Boolean,  
    var nome: String,  
    var dataNascimento: String,  
    var sexo: SexoUsuario,  
    val email: String,  
    var senha: String,  
    val tipoUsuario: TipoUsuario,  
    var cnpj: String? = null,  
    var razaoSocial: String? = null,  
    var nomeFantasia: String? = null  
)
```

25 Usos

```
enum class TipoEvento {  
    1 Uso  
    SOCIAL, CORPORATIVO, ACADEMICO, CULTURAL_ENTRETENIMENTO, RELIGIOSO, ESPORTIVO,  
    1 Uso  
    FEIRA, CONGRESSO, OFICINA, CURSO, TREINAMENTO, AULA, SEMINARIO, PALESTRA, SHOW,  
    1 Uso  
    FESTIVAL, EXPOSICAO, RETIRO, CULTO, CELEBRACAO, CAMPEONATO, CORRIDA, OUTRO
```

```
}
```

5 Usos

```
enum class ModalidadeEvento { PRESENCIAL, REMOTO, HIBRIDO }
```

16 Usos

```
data class DadosEvento(  
    val id: Int,  
    val organizadorEmail: String,  
    var pagina: String,  
    var nome: String,  
    var descricao: String,  
    var diaInicio: Int, var mesInicio: Int, var anoInicio: Int,  
    var horaInicio: Int, var minutoInicio: Int,  
    var diaTermino: Int, var mesTermino: Int, var anoTermino: Int,  
    var horaTermino: Int, var minutoTermino: Int,  
    var tipo: TipoEvento,  
    var idEventoPrincipal: Int?,  
    var modalidade: ModalidadeEvento,  
    var capacidadeMax: Int,  
    var local: String,  
    var statusEvento: Boolean,  
    var precoIngresso: Double,  
    var aceitaEstorno: Boolean,  
    var taxaEstorno: Double = 0.0  
)
```

```
data class DadosIngresso(  
    val id: Int,  
    val idEvento: Int,  
    val emailUsuario: String,  
    var statusDisponibilidade: Boolean,  
    val valorPago: Double
```

Tópico 4: Operações do repositório

- Objetivo

Como mencionado anteriormente, o Repositorio.kt define o banco de dados do sistema criando listas e funções de buscas.

Tópico 4: Operações a serem apresentadas

- val listaUsuarios = mutableListOf<DadosUsuario>()
- val listaEventos = mutableListOf<DadosEvento>()
- val listaIngressos = mutableListOf<DadosIngresso>()

Estas operações criam a lista que pode ser modificada para armazenar (ou não armazenar) os dados de alguma entidade.

Tópico 4: Operações a serem apresentadas

- fun digitarDataHoje()
- fun definirDataHoje()

Estas operações, que são funções, respectivamente funcionam:

1. Permitindo o usuário digitar data/mês/ano para informar a data
2. Gravando o que é guardado nas variáveis da fun digitarDataHoje() em variáveis “gerais”

Tópico 4: Operações a serem apresentadas

- fun adicionarUsuario(usuario: DadosUsuario)
- fun buscarUsuarioEmail(email: String): DadosUsuario?
- fun buscarUsuarioLogin(email: String, senha: String): DadosUsuario?
- fun possuiEventos(usuarioEncontrado: DadosUsuario): Boolean

Estas operações, que são funções, respectivamente funcionam:

1. Adicionando usuário a lista
2. Buscando um usuário por email
3. Buscando um login válido
4. Buscando se um usuário possui um evento (retornando verdadeiro ou falso)

Tópico 4: Operações a serem apresentadas

- fun adicionarEvento(evento: DadosEvento)
- fun buscarEventoid(id: Int): DadosEvento?
- fun buscarEventosDisponiveis()
- fun buscarEventosOrganizador(email: String): List<DadosEvento>

Estas operações, que são funções, respectivamente funcionam:

1. Adicionando evento a lista
2. Buscando um evento por ID
3. Listando eventos disponíveis
4. Listando eventos de um organizador com base num email de um usuário

Tópico 4: Operações a serem apresentadas

- fun adicionarIngresso(ingresso: DadosIngresso)
- fun buscarIngresso(id: Int): DadosIngresso?
- fun buscarIngressosUsuario(email: String): List<DadosIngresso>
- fun proximoIngresso(id: Int): Int
- fun ordenarIngressosUsuario(email: String): List<DadosIngresso>
- fun contarIngressosVendidos

Estas operações, que são funções, respectivamente funcionam:

1. Adicionando ingresso a lista
2. Buscando um ingresso por ID
3. Listando ingressos de um usuário
4. Buscando o número (ID) de um próximo ingresso
5. Listando ingresso do usuário e ordenando
6. Contando ingressos já vendidos

Tópico 5: Funções de componentes e dois formulários criados com elas

- Objetivo

As funções de componentes são extremamente úteis para um sistema. Elas colaboram ajudando os programadores a fazerem menos validações repetidamente ao longo do código. No Dendê Eventos, existem quatro: `readInt`, `readDouble`, `readString` e `printTable`.

Tópico 5: readInt e dois formulários criados com ela

A readInt faz leitura somente de números inteiros e pode definir um intervalo de números aceitos naquele formulário. Sua aplicação está em locais como:

```
val cadastroEstorno = readInt("Digite opção: ", "ERRO: Opção inválida. Tente novamente.", 1..2)
```

```
val confirmarCompra = readInt("Digite opção: ", "ERRO: Opção inválida. Tente novamente.", 0..1)
```

O funcionamento é simples: define variável, define mensagem, define mensagem de erro e define o intervalo de valores aceitos.

Tópico 5: readDouble e dois formulários criados com ela

A readDouble faz leitura de números racionais e pode definir um intervalo de valores aceitos naquele formulário. Sua aplicação está em locais como:

```
val cadastroPreco = readDouble("Digite preço: ", "ERRO: Preço inválido. Tente novamente.", 0.0)
```

```
val cadastroTaxa = readDouble("Digite Taxa (%): ", "ERRO: Taxa inválida. Tente novamente.", 0.0, 100.0)
```

O funcionamento é simples: define variável, define mensagem, define mensagem de erro e define o intervalo de valores aceitos.

Tópico 5: `readString` e dois formulários criados com ela

A `readString` faz leitura de qualquer texto e pode definir um tamanho mínimo de dígitos exigidos naquele formulário. Sua aplicação está em locais como:

```
val senhaInserida = readString("Digite senha: ", "ERRO: Senha mínima 8 dígitos. Tente novamente.", 8)
val cnpjInserido = readString("Digite CNPJ: ", "ERRO: CNPJ inválido. Digite 14 números.", 14)
```

O funcionamento é simples: define variável, define mensagem, define mensagem de erro e o tamanho mínimo de texto desejado.

Tópico 5: printTable e dois formulários criados com ela

A printTable faz exibição de listas e pode se adaptar ao espaço que cada lacuna ocupa na tabela. Sua aplicação está em locais como:

```
printTable("FEED DE EVENTOS", colunas, linhas)
```

```
printTable("SEUS INGRESSOS:", colunas, linhas)
```

Tópico 6: Resultado em Main.kt

- Objetivo

Para concluir a refatoração, o código da *fun main()* fica enxuto.

- Conteúdo

O que fica, por fim, no fluxo principal do programa são as chamadas de funções e os laços de repetições (loops) de menus.

Pergunta ao professor

- Qual é a melhor maneira de estruturar o slide? Mais (tópicos 1-3) ou menos (tópicos 4-6) exemplos com prints de código?

A equipe ficou com uma dúvida em relação a exemplificação do “trecho de código” necessário ao tópico proposto.

Referências

- [OAT 1 - Descobrindo Kotlin - Google Docs](#)
- [GitHub/lucassacramento/dende-eventos-londres/apresentacao_oat1_londres.pdf](#)
- [OAT 2 - Refatorando o código - Google Docs](#)
- [ATIVIDADE DE SONDAÇÃO 4 - Google Docs](#)
- [Lista de Exercícios 5: Introdução a Funções - Google Docs](#)